

합성곱 신경망(CNN; Convolutional Neural Network)

MLP의 한계

Convolutional (CONV) Layer 등장

합성곱 신경망의 구조 및 작동 방식

합성곱(Convolution)

필터(Filter)와 커널(Kernel)의 차이

스트라이드(Stride)

한 개 층에 큰 필터 쓰기 vs. 여러 개 층에 작은 필터 쓰기

패딩(Padding)

ReLU Layer

풀링(Pooling)

CNN의 일반적인 구조

CNN의 특징

CNN의 응용

요약

합성곱 신경망(Convolutional neural network, CNN)은 시각적 영상을 분석하는 데 사용되는 다층의 피드-포워드적인 인공신경망의 한 종류이다. 딥 러닝에서 심층 신경망으로 분류되며, 시각적 영상 분석에 주로 적용된다. 또한 **공유 가중치 구조**와 **변환 불변성** 특성에 기초하여 변이 불변 또는 공간 불변 인공 신경망 (SIANN)으로도 알려져 있다. 영상 및 동영상 인식, 추천 시스템, 영상 분류, 의료 영상 분석 및 자연어 처리 등에 응용된다.

- 위키백과 (합성곱 신경망)

- 합성곱 신경망은 주로 picture, image 분석에서 많이 사용되는 신경망 구조
- 합성곱을 식으로 설명하면 두 함수가 있고, 둘 중 하나를 뒤집은 후에 곱한 결과를 적분함

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

- 이는 함수에 들어가는 값들이 연속적인 경우이고, 합성곱 신경망에서의 합성곱은 각각의 값이 따로 따로, 즉 이산적이기 때문에 아래와 같이 표현 가능함

$$(f * g)(t) = \sum_{\tau} f(\tau)g(t - \tau)$$

- 결론적으로 간단히 말하면 짝이 맞는 숫자 둘($f(\tau)$ 와 $g(t-\tau)$)이 만나 곱해진 값들을 한 곳에 모아 합을 구해주는 것으로 이해하면 됨

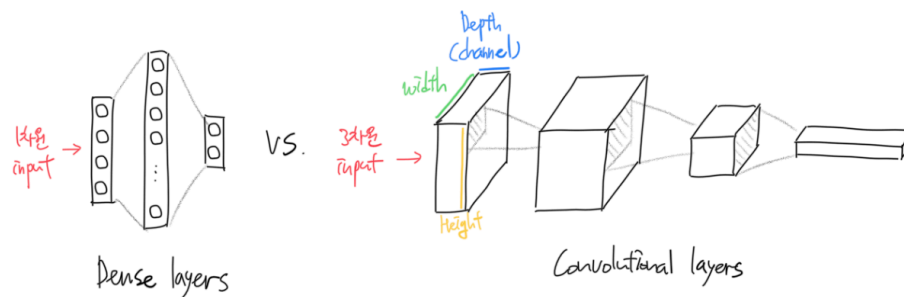
MLP의 한계

- MLP에서는 별도의 feature engineering 없이 raw feature를 곧바로 입력층에 넣음
 - 데이터의 차원이 크지 않은 경우에는 raw feature를 전부 사용해도 모형의 파라미터 수가 크지 않았기 때문에 학습 과정에서 문제가 생기지 않았고 성능도 좋았지만, 이미지와 같은 데이터에서는 문제가 달라진다.
 - 이미지 데이터를 예시로 들어보자. 가로 48픽셀, 세로 48픽셀의 컬러(RGB) 이미지 데이터를 생각해보자. 이 이미지의 각각의 픽셀을 하나의 feature로 사용해서 MLP의 입력층에 넣을 수 있을 것이다. 이렇게 feature engineering 없이 각 픽셀의 raw value를 전부 입력으로 넣는다면, 뉴럴넷이 감당해야 하는 feature 차원수는 $48 \times 48 \times 3 = 6912$ 가 된다. 겨우 가로세로 48픽셀밖에 안되는 저화질 이미지였는데도 feature 차원이 7000에 달한다. Feature 갯수가 이렇게 많아지게 되면 컴퓨팅 속도가 저하되고 오버피팅의 위험이 생길 수 있음
- MLP에서는 각 샘플이 등장한 맥락을 고려하지 않음
 - MLP에서는 샘플들이 정렬되어 있는 순서가 중요하지 않았다. 그러나 주식 일별 종가, 문장 속 단어와 같은 데이터에서는 각 샘플의 앞 뒤로 어떤 샘플들이 있었는지의 정보, **문맥**도 중요하다. 이 경우엔 MLP만으로는 충분히 좋은 성능을 발휘할 수 없음

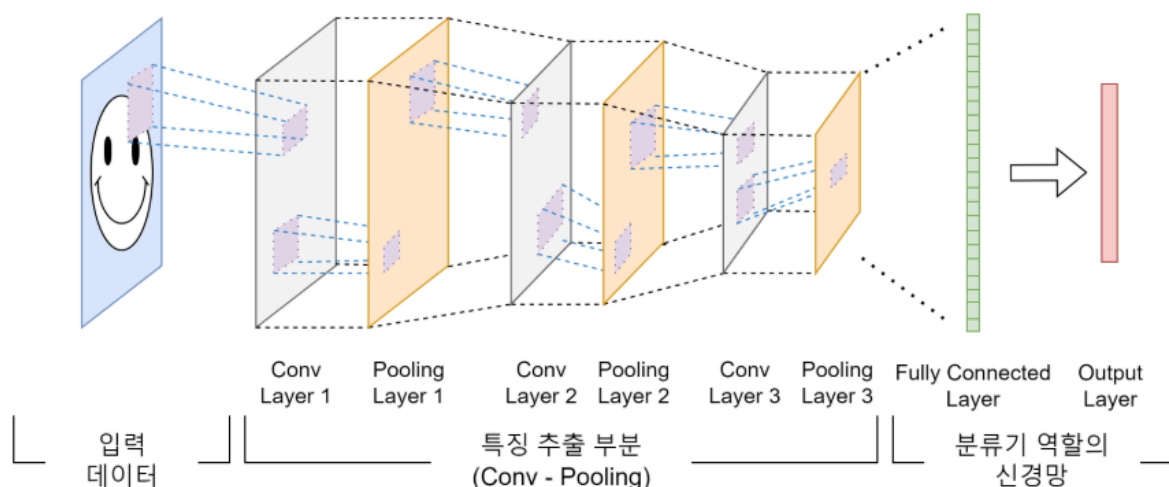
Convolutional (CONV) Layer 등장

- 이미지의 각 픽셀을 그대로 feature로 사용하는 것은 좋지 않은 방법
- 각 이미지로부터 독특한 특성만을 뽑아내어 feature로 사용한다면 더 좋은 성능을 낼 수 있음
- 이 과정을 feature extraction이라고 하고, 이미지에서 feature extractor로 동작하는 레이어가 바로 convolutional layer
- Convolutional layer는 두 가지 가정을 전제함
 - Local connectivity

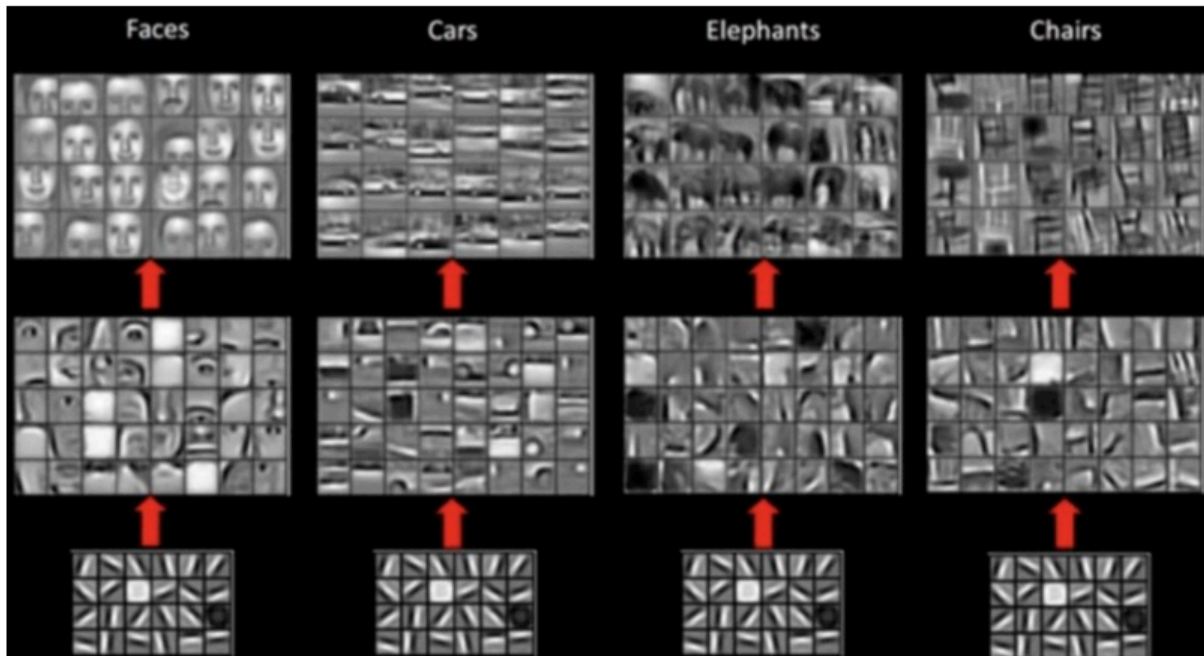
- 이전 레이어의 모든 뉴런과 연결되는 MLP에서와는 달리 CONV layer는 이전 레이어의 뉴런 중 일부와만 연결됨. 즉 이미지의 모든 픽셀과 연결되는 대신 이 이미지 중 일부분의 픽셀과만 연결
- Spatial invariance
 - 이미지의 한 부분의 데이터 분포는 다른 부분에서의 데이터 분포와 다르지 않다. 즉, 이미지의 분포적인 특성은 어느 부위에서든 동일하다고 가정
- MLP의 Dense layer에서 각 입력 샘플을 1차원의 벡터 데이터로 보았다면, Convolutional layer는 입력 샘플을 3차원의 텐서(Tensor) 데이터로 인식함. Convolutional layer는 입력 3차원 데이터를 적절히 변형해서 또다른 3차원 데이터를 출력한다



합성곱 신경망의 구조 및 작동 방식



- 입력 데이터(이미지)가 들어오면 특징 추출 부분에서 합성곱 연산 및 풀링 등의 과정을 거쳐 이미지의 특징을 잡아냄
- 신경망의 초기에는 저수준, 즉 이미지의 아주 작고 세부적인 특징부터 찾는다. 그리고 층이 깊어질수록 고수준, 즉 이미지의 전체적인 특징을 찾아냄

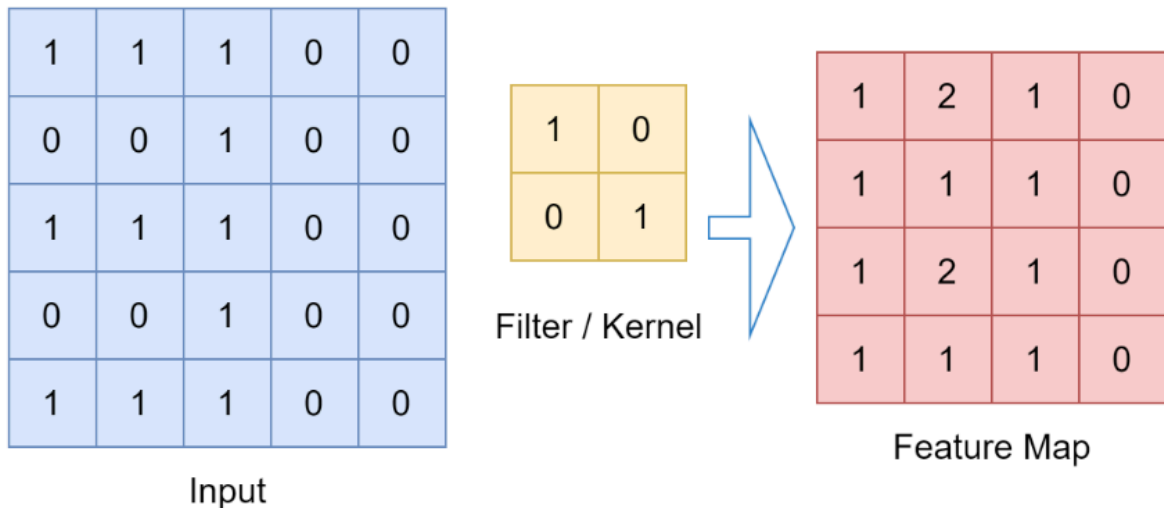


- 처음에는 가로, 세로, 대각선 정도의 특징부터 잡기 시작
- 점차 (사람 얼굴인 경우) 눈, 코, 입 등을 구분
- 마지막에는 전체적인 얼굴 윤곽을 추출하는 함

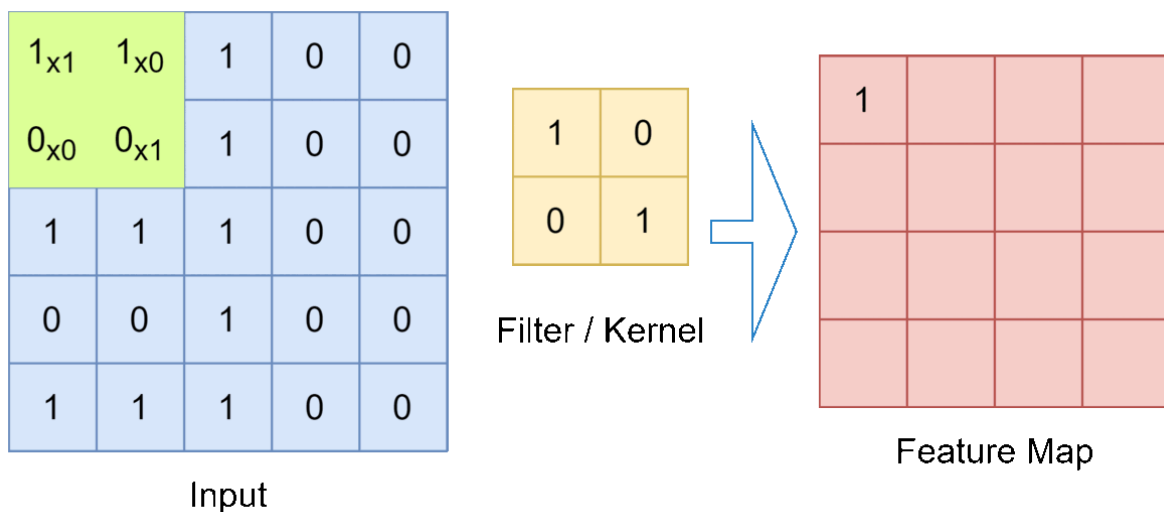
합성곱(Convolution)

합성곱 신경망에서의 합성곱은 결국 짝이 맞아 서로 곱해진 두 숫자의 커플들을 한데 모아 합한 것

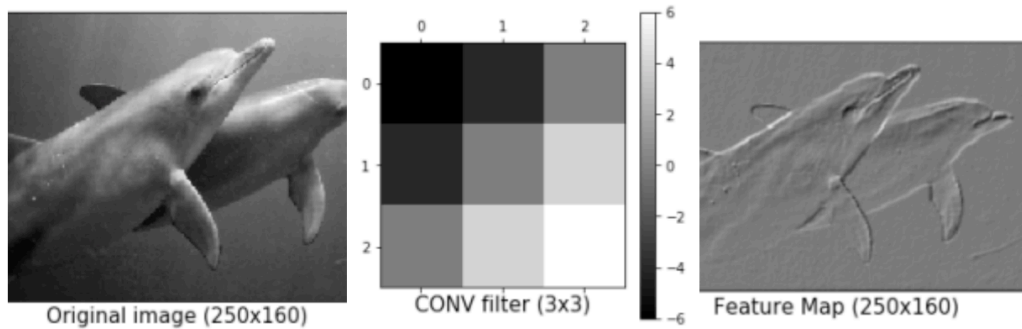
$$(f * g)(t) = \sum_{\tau} f(\tau)g(t - \tau)$$



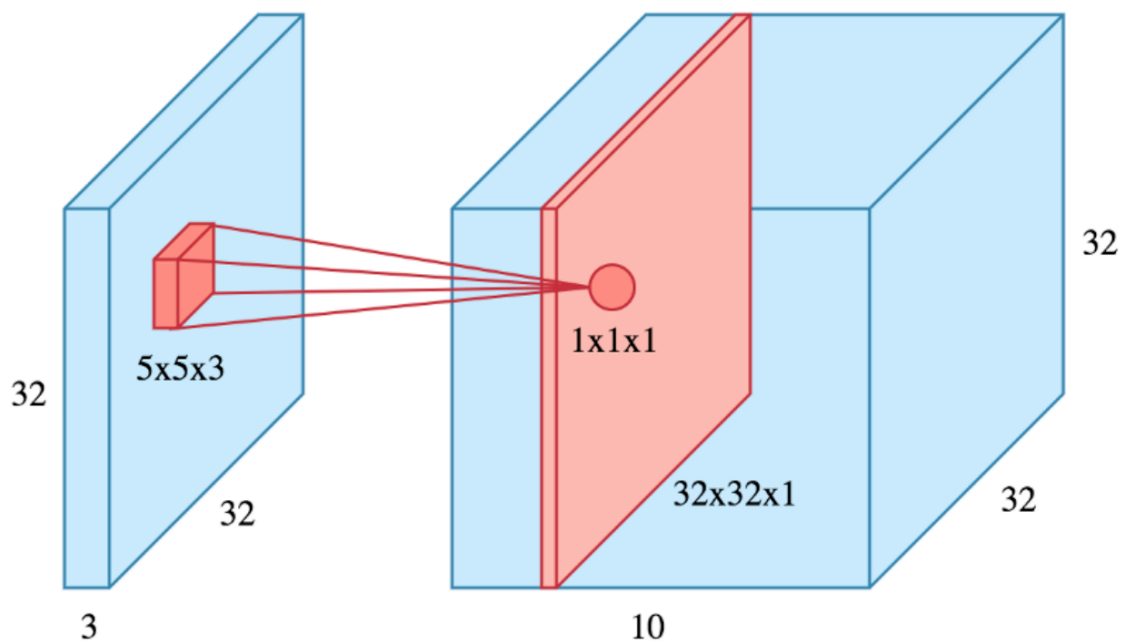
- 우선 5X5 사이즈의 그레이 스케일(흑백) 이미지가 있다고 가정
- 이 이미지 위를 2X2 사이즈의 필터(Convolution Filter)가 왼쪽 위부터 시작해서 몇 칸 단위(스트라이드, stride)로 차례차례 돌아다닌다고 가정
 - 한 칸씩 움직인다고 하면 가로로 네 번, 세로로 네 번 해서 총 16번



- 이때 매번 움직일 때마다 필터 위의 숫자와 이들 각각의 위치에 해당하는 이미지 위의 숫자를 곱한 후, 결과를 전부 합함.
- 이렇게 해서 각 결과를 계산된 자리대로 배치해서 내놓은 것을 **Feature Map**이라고 함 (이 과정에서 **Filter의 가중치가 학습**이 된다.)
- Feature Map에 활성화 함수를 적용한 결과를 Activation Map이라 하고, 여기에 편향 (bias)까지 합해준 것이 합성곱 층의 최종 결과물(Output)이다.

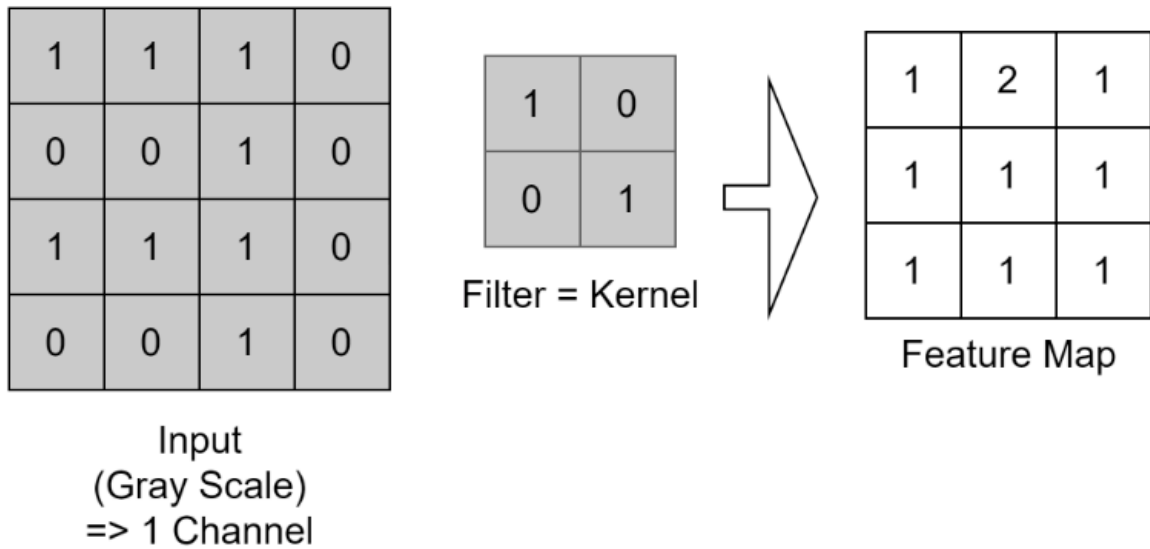


- 위 이미지처럼 우상에서 좌하로 향하는 사선 모양의 convolution filter는 이미지 내에서 사선 모양이 있는 부분(주로 이미지의 윤곽선)을 보면 큰 값을 출력한다. 위 돌고래 이미지에서 우상좌하 사선 모양의 윤곽선이 도드라지게 추출된 것을 확인할 수 있음
- 단, 실제 ConvNet에서는 필터를 우리가 직접 정해주지 않고 알고리즘이 데이터를 바탕으로 문제 해결에 적합한 형태가 되도록 학습해나감
- 필터 하나 당 하나의 feature map이 만들어지므로, 필터 n개를 사용하는 CONV layer의 출력은 $\text{depth}=n$ 을 갖는 텐서가 된다. 만약 32×32 짜리 컬러($\text{depth}=3$) 이미지 하나가 필터 10개를 사용하는 CONV layer를 통과하면 그 출력 depth 는 10이 된다.

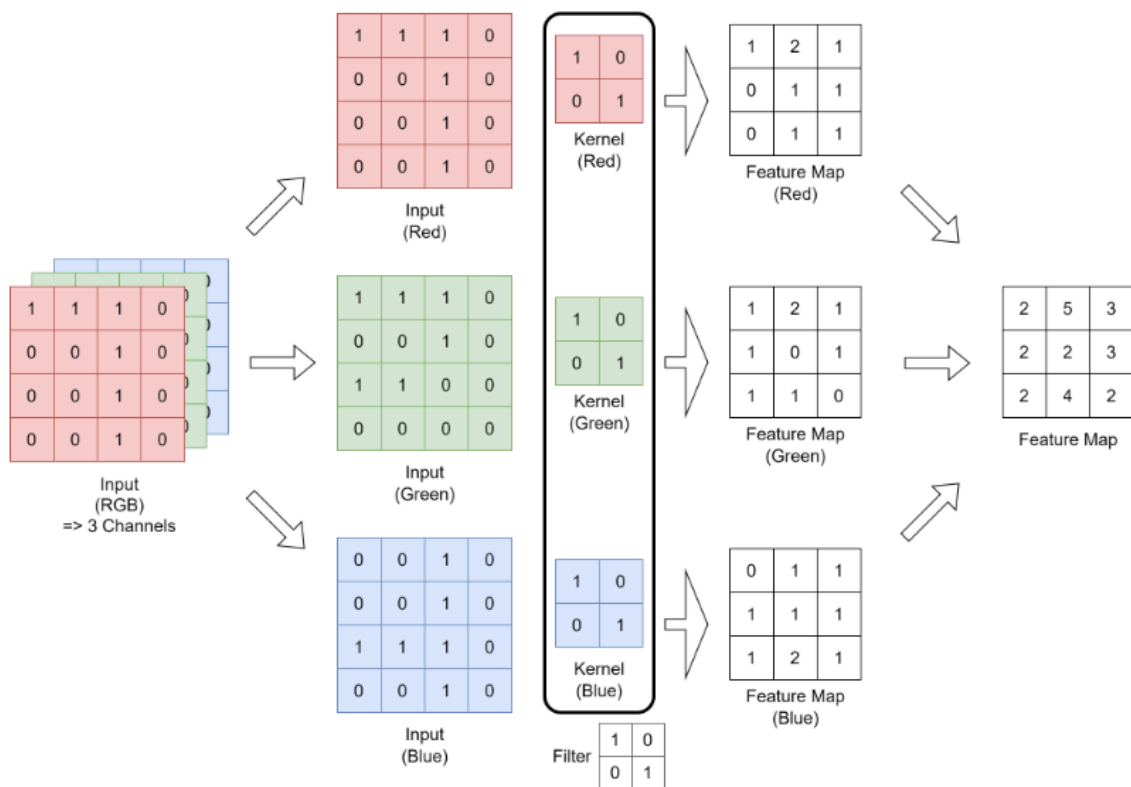


필터(Filter)와 커널(Kernel)의 차이

- 이미지가 그레이 스케일(채널(Channel)이 1개)인 경우 필터(Filter)는 커널(Kernel)과 같음



- 이미지가 컬러(RGB → 채널이 3개)인 경우 하나의 필터 안에 3개의 커널이 있음
 - 즉, 각 채널마다 1개의 커널이 있고, 필터는 커널의 집합이다.



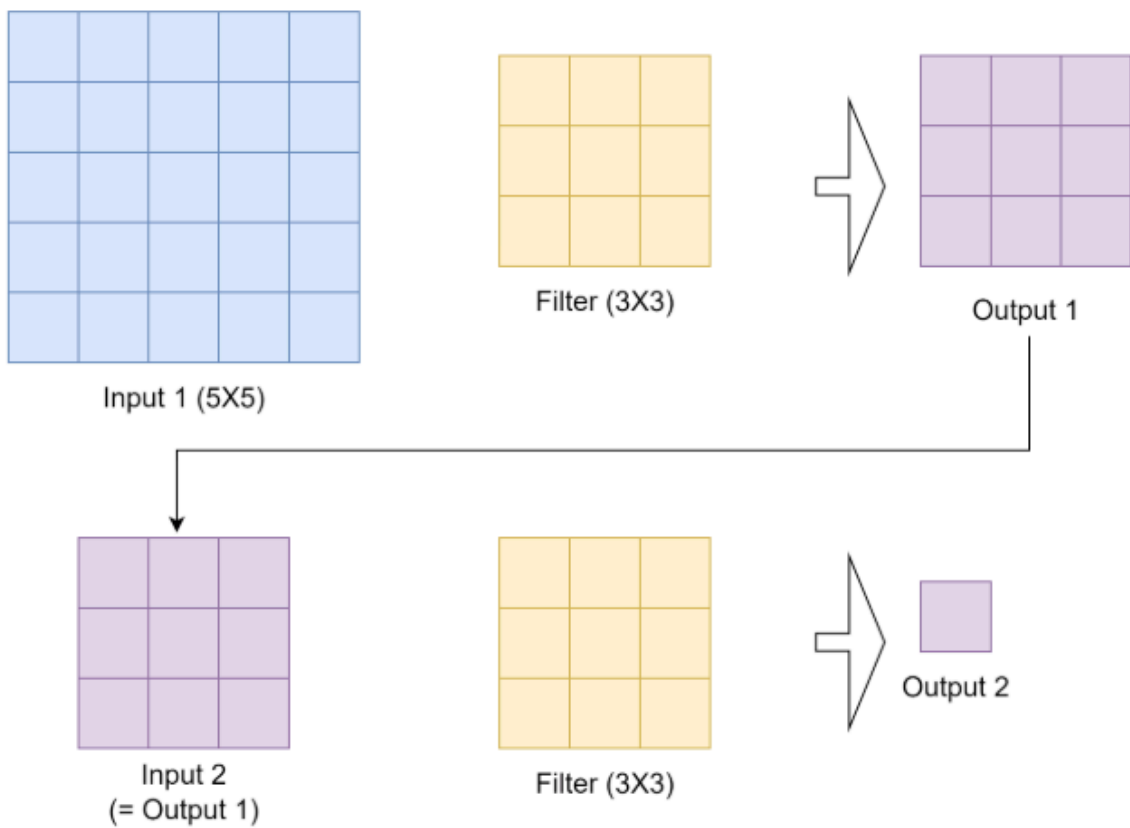
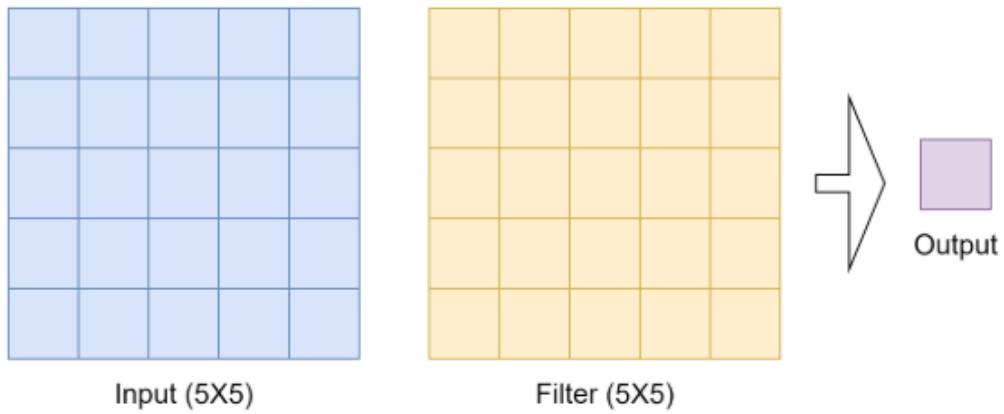
스트라이드(Stride)

- '몇 칸 단위'를 의미하며 몇 칸 단위로 필터가 돌아다니게 할지 정함

- 영어로 걸음, 보폭이라는 의미를 갖고 있음

한 개 층에 큰 필터 쓰기 vs. 여러 개 층에 작은 필터 쓰기

- 큰 필터 1개를 쓰기보다는 작은 필터 여러 개를 쓰는 것이 성능이 좋음
 - 학습되어야 할 가중치 파라미터의 수가 적어지고 일반적으로 더 나은 성능을 보이기 때문임.
- 예를 들어 5X5와 3X3 크기 필터가 있다면 입력 이미지의 크기가 5X5라고 했을 때 5X5 필터는 1개 층, 3X3 필터는 2개 층을 거치면 같은 결과를 얻음



- 5X5 필터는 학습되어야 할 가중치 파라미터의 수가 25개이고, 3X3 필터는 2개 층이므로 $3 \times 3 \times 2 = 18$ 개임
- 필터의 크기를 작게, 그리고 여러 층으로 만든 것이 큰 필터를 사용하는 것보다 적은 비용 필요

패딩(Padding)

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

- 패딩은 이미지의 외곽을 어떤 값으로 둘러싸는 것을 말함
- 만약 패딩을 주지 않고 합성곱을 진행하면 이미지의 테두리 부분은 안쪽에 비해 연산 과정에 덜 포함됨
 - 이는 달리 말하면 해당 부분의 정보가 제대로 반영되지 못한다는 의미
- 패딩은 전체 이미지의 값을 충분히 활용하기 위해서 사용함
- 입력 데이터의 크기가 패딩에 의해 변하게 되면서 Feature Map의 크기도 조절 가능함
 - 예제 이미지에서 만약 5X5 사이즈 이미지에 3X3 필터를 적용하면 3X3 Feature Map이 나올 것이다.
 - 패딩을 적용하여 7X7 사이즈가 되게끔 하였고, 여기에 3X3 필터를 적용하니 5X5 사이즈의 Feature Map이 나옴
 - 원본 이미지의 사이즈 5X5와 동일한 크기의 Feature Map을 출력으로 얻을 수 있음
- 패딩의 값으로는 보통 0을 사용하는데, 이를 제로 패딩(Zero-Padding)이라고 함
- 패딩은 인공 신경망이 이미지의 외곽을 인식하게끔 만드는 효과도 있다.

※ 필터 크기(Filter Size), 패딩(Padding), 스트라이드(Stride)와 Feature Map 크기의 관계

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

N_{in} : 입력되는 이미지의 크기(=특성 수)

N_{out} : 출력되는 이미지의 크기(=특성 수)

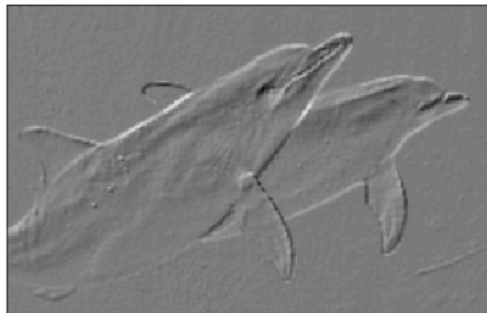
k : 합성곱에 사용되는 필터(=커널)의 크기

p : 합성곱에 적용한 패딩 값

s : 합성곱에 적용한 스트라이드 값

ReLU Layer

- CONV layer를 통해서 이미지로부터 추출한 feature map에서 양의 값만을 활성화시키는 레이어
- ReLU layer에서는 feature map에서 특히나 두드러지는 특징을 다시 추출
- 사실 layer라기 보단 CONV layer의 activation function으로 ReLU를 사용한 것으로 이해



CONV



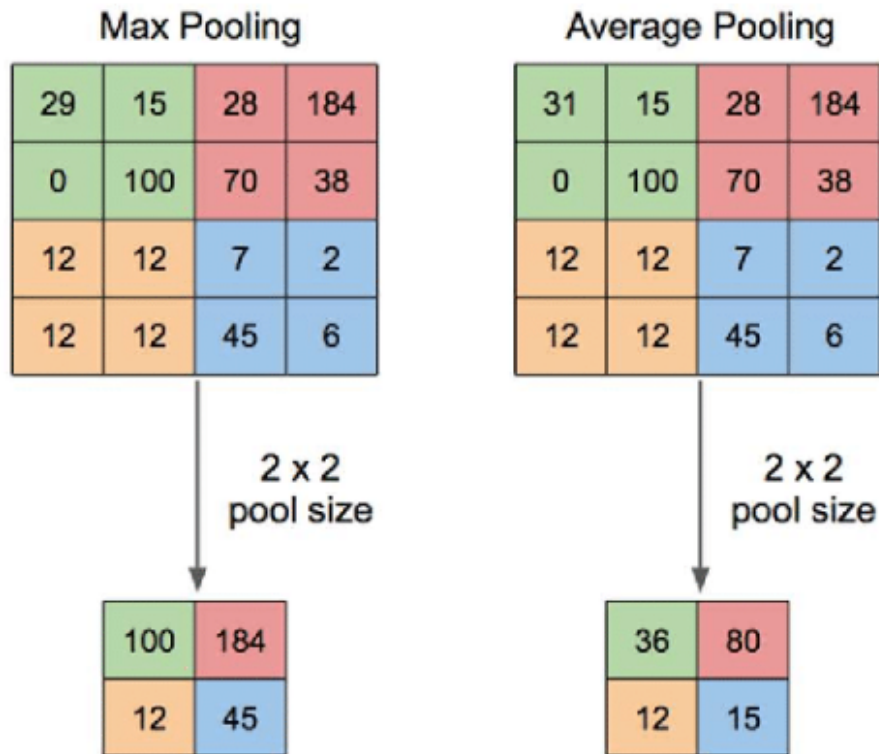
CONV>RELU

- ReLU activation 적용 후 우상좌하 사선 모양의 윤곽선이 매우 잘 추출됨

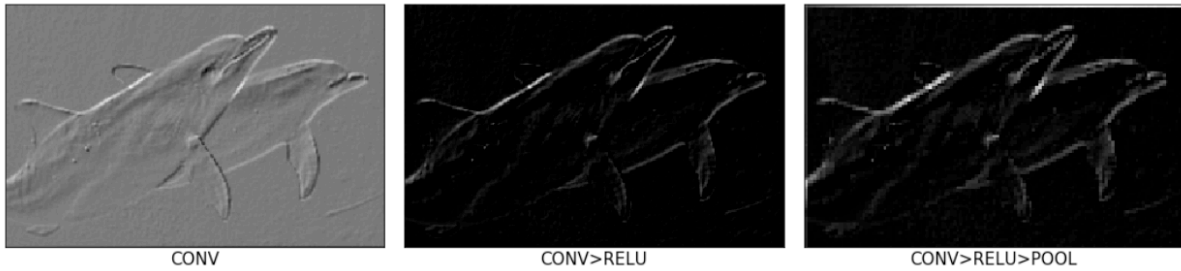
풀링(Pooling)

- CONV layer는 원본 이미지의 가로세로 크기를 유지하면서 depth를 수백~수천 수준으로 키우기 때문에 메모리를 많이 사용하는 레이어임

- 이 때문에 이미지와 feature map의 가로세로 크기를, 그 특징은 유지하되 적절한 수준으로 다운샘플링하여 메모리를 절약할 필요가 있음
- 다운샘플링 방법 중 ConvNet에서는 **pooling**을 사용함



- CNN에서 풀링을 거치면 이미지의 주요한 특징이 요약된 결과를 얻을 수 있음
- 위 그림에서 보여지듯 일정한 간격으로 영역을 나눈 후, 그 영역의 특징을 가장 잘 설명하는 값을 가져옴
- 여기서 이 값으로 영역 내 최대값을 갖고 올지 평균값을 갖고 올지에 따라 Max Pooling과 Average Pooling으로 구분함
 - 주로 Max Pooling이 사용
- 풀링을 통해 결과물, 즉 다음 층으로 향하는 데이터의 입력 크기(input size)가 축소되어 메모리 크기도 축소되는 효과가 있다.
- 파라미터 수를 줄여주는 효과도 있어 과적합을 방지할 수도 있다(Dropout 같은 느낌).
- 풀링 층은 따로 학습되는 가중치가 없고, 채널 수도 변하지 않는다.
- 돌고래 이미지의 예시에 2x2 max pooling (stride 2)을 적용한 결과, 아래와 같이 feature map의 특징은 유지하면서 이미지 크기를 1/4로 축소할 수 있음

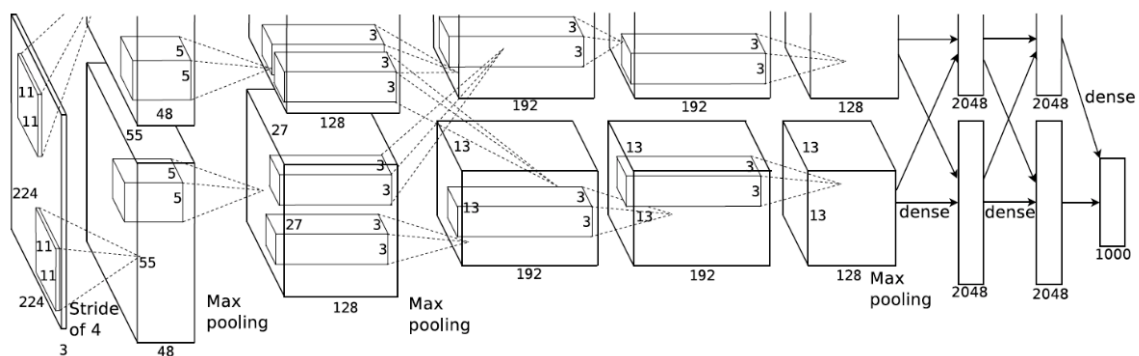


CNN의 일반적인 구조

- 일반적으로 ConvNet의 구조는 다음과 같이 표현할 수 있음

$$\{(CONV + RELU) \times n + POOL\} \times m + (Dense \times k)$$

- 여기서 CONV + RELU + POOL은 feature extractor 역할을 하고 Dense layer는 classifier 또는 regressor 역할을 한다. 이미지 분류 문제를 예시로 들자면,
 - CONV + RELU + POOL은 인풋 이미지로부터 분류에 사용할만한 특징적인 feature를 추출
 - Dense layer는 추출된 feature를 입력받아 이미지를 적당한 카테고리로 분류
- 얕은(입력층에 가까운) CONV layer일수록 단순한 특성 - 윤곽선, 음영 등을 추출
- 깊은(출력층에 가까운) CONV layer일수록 복잡한 특성 - 동물 모양, 특정 색상 조합 등을 추출
- 2012년 ILSVRC에서 우승한 AlexNet이 유행시킨 구조
 - 딥러닝을 사용해서 효과적으로 이미지를 분석할 수 있다는 것을 보여준 첫 사례



AlexNet

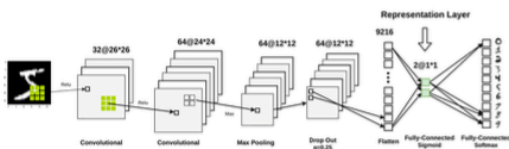
- AlexNet은 5개의 CONV layer와 3개의 dense layer를 사용하여 최종적으로 1000개 카테고리에 대한 softmax 값을 출력

CNN의 특징

- Feature의 위치에 구애 받지 않음
 - 한 이미지를 좌측 상단에 노란색 픽셀(특징)을 가진 경우와 우측 하단에 노란색 픽셀을 가진 경우를 만든 다음, 이 둘을 max pooling 처리를 계속 반복하면 두 위치가 달랐던 특징들이 결국 pooling 이후에는 동일한 위치 또는 근처로 모이게 되는 사례입니다. 이로써 CNN의 pooling 과정이 feature의 위치와 독립적인 "위치 불변성(Translation Invariance)"을 만들어냄
- 같은 입출력을 갖는 FC Layer에 비해 더 적은 weight를 가짐
- 병렬 계산 구성이 쉬우므로, GPU에서의 연산이 매우 빠름

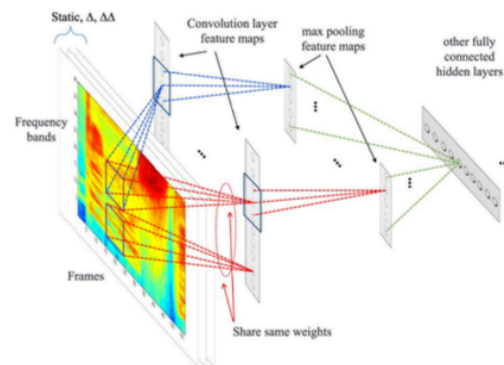
CNN의 응용

Computer Vision



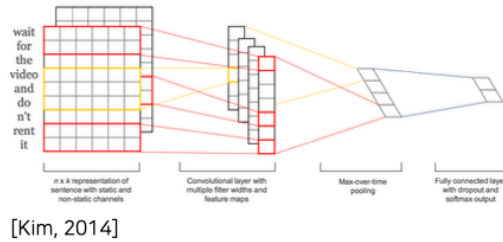
[Mirzaei et al., 2014]

Speech Recognition

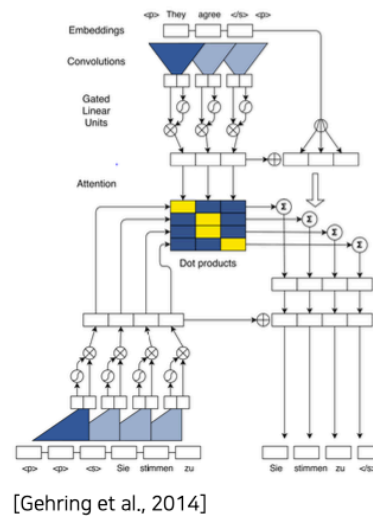


[Abdel-Hamid et al., 2014]

Text Classification



Machine Translation



Time Series

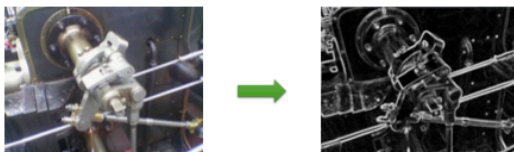


요약

Convolutional Neural Networks

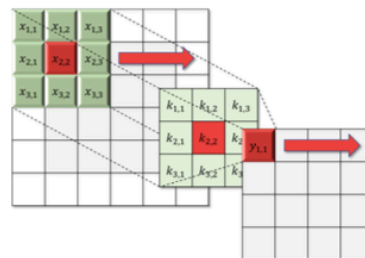
Before Deep Learning

- Convolution filter를 활용하여 hand-crafted feature 추출



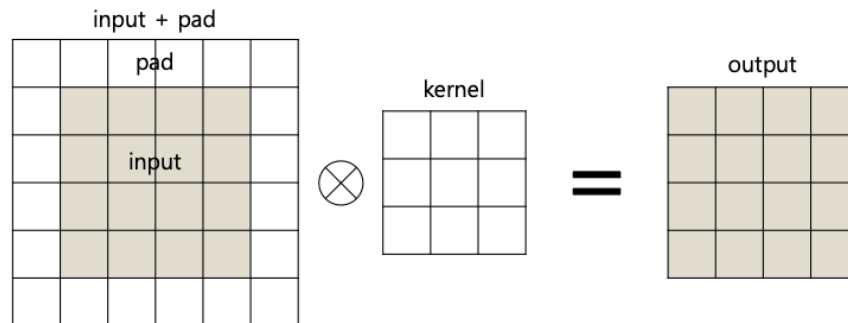
In Deep Learning

- $x \rightarrow y$ 의 관계의 학습에서 필요한 feature를 추출하기 위한 convolution filter를 자동 학습



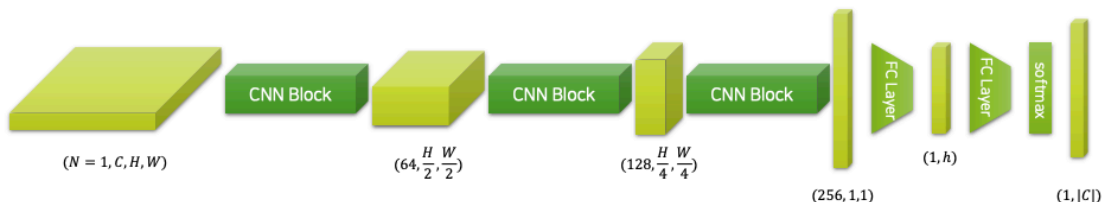
vs Fully-connected Layer

- 매우 빠르고 적은 weight parameter를 가짐
- 입출력 크기가 계산이 까다로워, 네트워크 구성이 쉽지 않다.
 - 3x3 커널을 활용하면 그래도 좀 낫다.



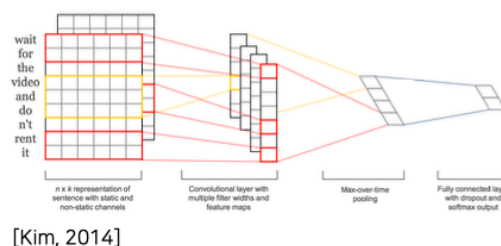
How to design CNN Architecture

- CNN Block
 - 1) 3x3 Convolution Layer
 - 2) ReLU
 - 3) Batch Normalization
 - 4) 3x3 Convolution Layer (+ with Stride size (2x2))
 - 5) ReLU
 - 6) Batch Normalization
 - 7) (+ Max-pooling if no stride)



Wrap-up

- Computer vision 분야에서 CNN을 떨 수 없는 존재
- 패턴 인식에서 워낙 월등함을 보여주므로, NLP 등의 다른 분야에도 활발히 사용됨



[Kim, 2014]